

Passive Pre-Association Wi-Fi Access-Point Selection

U.K.Arvindan
EP21B006

Abstract— A Wi-Fi client in overlapping coverage must select an access point (AP) before it can directly measure the service that each AP would provide. Signal strength describes link quality, but not contention or load, and may therefore favor a nearby but congested AP. This project investigates whether a client can make a better decision using only passive observations collected before association without assistance from the network.

We define a controlled ns-3 simulator of multi-AP deployments to investigate this. The simulation of each topology of devices is replayed n times to a candidate STA to obtain the throughputs offered by each AP.

I. PROBLEM FORMULATION

Throughput depends on channel contention, competing stations, transmission rates, retries, and the airtime consumed by neighboring basic service sets. Received signal strength reflects such effects poorly if at all. Those effects do, however, leave traces a client can hear without transmitting: the frames themselves, and the time the medium spends busy. This project asks whether such traces are enough: *can a client choose the AP that will provide the greatest throughput using only what its radio can passively observe before association?*

Existing load-aware methods [1], [2] make their load estimate after association, from AP-provided reports, or from a network controller. A client making its first choice has none of these: only beacons and some data frames, in which AP distance and offered traffic appear only through their effects.

Let $\mathcal{A} = \{a_1, \dots, a_{N_{AP}}\}$ be the APs in one deployment and let O be the candidate station's passive observation before it joins any AP. If y_i is the throughput obtained after joining a_i , the desired decision is

$$a^* = \arg \max_{a_i \in \mathcal{A}} y_i, \quad (1)$$

using a decision function whose inputs are derived only from O .

We tackle this problem by predicting the value of y_i for each a_i .

It is worth noting that models that simply output a ranking of the available choices are viable as well. In the simulated data we ensure that we have true counterfactuals by deliberately verifying O to be identical across all N_{AP} runs of a given scenario. In a live network however, only one y_i is ever measurable: the station joins a single AP and observes that AP's throughput, while the values of y_j for every AP it did not join are not easily recoverable. For broader applicability,

and adaptation to reality, choice models are not investigated extensively here.

II. SIMULATOR DESIGN AND ASSUMPTIONS

A. Simulation Environment

The simulator is built on ns-3.45 and its Wi-Fi models [8]. The deployment is static and single-floor with no walls, the environment TGax assumes for its indoor small-BSS scenario [5]. All stations are immobile and remain at their seeded positions. Radios are homogeneous IEEE 802.11n devices at 5 GHz on 20 MHz channels with Minstrel-HT rate control [8].

Every radio is an ns-3 `SpectrumWifiPhy` attached to one shared `MultiModelSpectrumChannel`, on which a log-distance loss (exponent 3.0, with 46.6777 dB of reference loss at 1 m) is followed by Nakagami fading.

`SpectrumWifiPhy` is chosen over the simpler `YansWifiPhy` as the Yans model simulates exactly zero cross channel interference by treating each channel as an isolated layer. If `YansWifiPhy` is used, the channel occupancy given by CCA measurements becomes a powerful signal for models in a way that does not reflect reality.

B. Count, Topology and Channel occupancy of APs

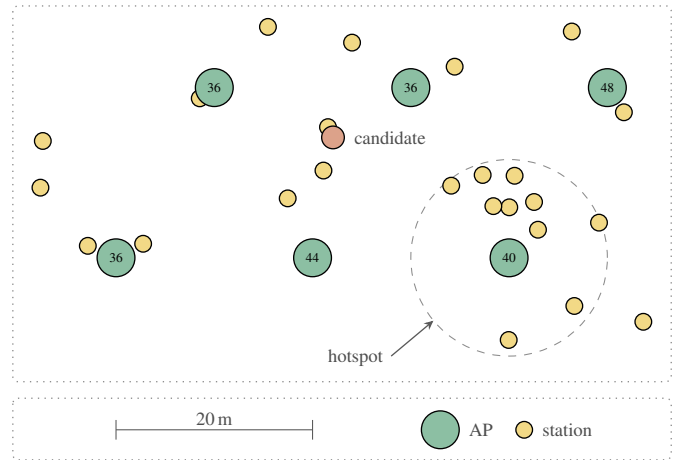


Fig. 1. An example simulated deployment.

Each scenario first draws 2, 3, 4, 6, or 8 APs. They sit on a triangular lattice 20 m apart, following [5], where it is reasoned that such a topology keeps the nearest-neighbor distance from varying with the number of APs. APs are confined to two rows of the lattice with the bottom row having $\min(2, \lceil \frac{N_{AP}}{2} \rceil)$ APs.

Each AP then draws a channel uniformly at random from channels 36, 40, 44, and 48. Random assignment biases the

data toward more in-channel contention and interference which could potentially make the problem harder than if we plan out the channels to be minimally interfering. However, TGax sets precedent for random assignment across four channels in its enterprise scenario, and across the non-overlapping set in its residential one [5], so this is the strategy adopted.

C. Count, Topology of background STAs

The total number of stations is decided by independently drawing a count of 3, 4, or 5 N_{AP} times and summing them.

A quarter of scenarios spread the stations uniformly. The rest declare one or more APs as "hotspots" around which stations are clustered. Such mixing of uniform and deliberately clustered regimes follows common practice in WLAN studies [3], [4], and is of particular import here because under uniform load every AP is about equally busy and the problem we examine collapses to signal strength in a fashion uncharacteristic of reality.

We draw the number of hotspots at random with fewer hotspots being more likely and the count capped at $\lceil N_{AP}/3 \rceil$. The area around a hotspot AP, a 10 m disc around it, is assigned a probability density 4 times that of non-hotspot areas. Each station's position is drawn from this distribution.

D. Association and Load of background STAs

Each station associates with the AP whose received power is highest. This power is calculated with a log-distance term, then perturbed by a shadowing term: a random sample from a Gaussian of 5 dB standard deviation. Without the perturbation, stations form exact Voronoi partitions, which no real deployment does; 5 dB is the indoor shadowing deviation TGax specifies [5].

Every station then draws its own offered load, independently, from a log-normal distribution with median 2.25 Mbit/s and shape 0.9, capped at 25 Mbit/s, for a mean of 3.37 Mbit/s. Traffic volumes are log-normal rather than Gaussian [7], and the right skew is what puts most stations on a light load and a few on a heavy one without needing a mixture. The quartiles span the mix an AP actually carries, from voice and browsing up through high-definition video.

E. Placement of candidate STA

The candidate station is placed uniformly at random over the same area as the background stations: the AP footprint plus a 10 m border.

F. Data Capture

Background traffic starts at $t = 0.5$ s. Each station's arrivals are Poisson at its own data rate, and its first packet is delayed by a phase drawn uniformly over one mean interval, without which, stations that share an AP and a rate transmit in lockstep for the whole run.

Passive recording runs from $t = 2$ s to $t = 8$ s, covering 6 s of loaded traffic. It starts 1.5 s after the traffic because Minstrel-HT needs about that long to converge, and until it does every scenario carries the same opening burst of retransmissions.

The candidate starts joining at $t = 8$ s, the moment recording ends. After joining, the candidate offers 100 Mbit/s in 1250-

byte packets until the run ends at $t = 20$ s. That rate keeps it backlogged, so its measured throughput reflects the service the network gives it rather than the limit of its own traffic generator.

G. Matched Counterfactual Replays

For each sampled topological arrangement of the candidate + environment, the experiment runs once per target AP. AP and station positions, background traffic, the candidate's position, and every pre-association device stay fixed; only the AP selected at $t = 8$ s changes. A `topologySeed` fixes the deployment: station placement, channel selection, and the load offered by each station. Independently, `candidateSeed` fixes the candidate STA's position, and `rngSeed` controls all other randomness such as fading, contention, rate adaptation, and arrival times of background packets.

Quantity	Distribution
AP count	uniform on {2, 3, 4, 6, 8}
AP channel	uniform on {36, 40, 44, 48}, iid per AP
Station count	sum over APs of uniform on {3, 4, 5}
Spatial regime	uniform with prob. 1/4, else hotspots
Hotspot count	$1 \dots \lceil N_{AP}/3 \rceil$, fewer more likely
Hotspot field	$4 \times$ station density inside a 10 m disc
Station load	log-normal, median 2.25 Mbit/s, shape 0.9, cap 25
Association	strongest received power + $\mathcal{N}(0, 5^2)$ dB
Candidate position	uniform over the station area

III. DATASET CONSTRUCTION

A. Recording and Projection

A deployed client scans across all channels that host APs in a rotating fashion. The simulation records more than any deployed client can be expected to observe by recording activity on every occupied channel simultaneously. We then project this complete record down to what a single scan would see.

Specifically, the candidate STA carries one receive-only scanner PHY per occupied channel, plus a separate PHY reserved for association. The scanner PHYs never transmit and never associate; the association PHY is parked on unused channel 149 until the decision time. At $t = 8$ s the scanner PHYs move to the parking channel, and the association PHY moves into the channel of the AP it intends to associate with.

Two traces are recorded. The monitor-sniffer trace holds every frame a scanner decodes, with the start and end of the transmission carrying it, frequency, BSSID, transmitter, frame category, beacon and retry flags, length, RSSI, airtime, and PHY rate. Frames aggregated into one transmission share their start and end timestamps.

The PHY-state trace gives the fraction of each 1 ms interval in which carrier sense marked the channel busy. This clear-channel-assessment (CCA) measurement registers energy a frame-only view misses: interference, and transmissions too distant to decode. Both thresholds are left at the ns-3 defaults: -62 dBm of received energy from any source, or -82 dBm from a signal whose preamble the PHY recognizes. Most driver stacks expose an equivalent channel-busy counter, though availability varies by platform.

The projection onto one passive radio is done as follows: One radio is assumed to rotate through the occupied channels in a certain randomly decided order, dwelling 110 ms on each with 0.15 ms to retune, and repeats the cycle for as much of the observation window as possible. A frame is kept only if its whole transmission falls inside a dwell after retuning: CCA intervals outside a dwell are discarded. A 110 ms dwell spans just over one 102.4 ms beacon interval, so each visit yields about one beacon from every AP on that channel.

RSSI is then perturbed by uniform error within ± 3 dB and quantized to 1 dB. Both steps run after ns-3 from a recorded scan seed, so scan policy and measurement noise can be varied without repeating a simulation.

B. Training Data

The Dataset used in this project comprised of 1000

IV. MODEL ARCHITECTURE

The model adapts a pretrained language model to this problem following the structure of NetLLM [9]: A learned encoder turns the observation into a sequence of vectors, a frozen decoder-only backbone reads that sequence as input embeddings, and only low-rank adapters [10] on its attention projections are trained.

Every model in this section scores the *discovered APs* of a scan: those whose beacons it decoded at least once. An AP never heard leaves nothing in the observation to read from, so the arg max of Section I is in practice taken over the discovered APs rather than over every AP in the deployment. This is a limit of passive observation rather than of the models, and it costs little, since an AP too distant for its beacons to arrive is unlikely to be the one that would have served best.

A. Embedding

The observation window contains several thousand frames, and carrier sense datapoints making attending over the scan frame by frame impractical. Further, several meaningful signals lay as aggregates or counts which would not be best captured by such a model.

Thus, we undertake the following process to vectorize our observation: First, the scan is cut into bins over time. Each bin is chosen to be of 110 ms, such that it coincides with channel dwells, keeping same-channel frames together. For each bin, we emit several *cells*. One cell is emitted per discovered AP: each of these only contains frames that carry its BSSID. We also emit one channel cell with all the frames of the dwell.

The model then learns a representation for each cell by allowing frames within the cell to attend amongst themselves. Frames carry 10 numerical values and a categorical flag; 9 numbers and the category are embedded into a 48-dimensional space with a 2 layer MLP. Non-linearity is needed here to capture relations between the frame characteristics, some of which are boolean, for attention to better relate frames to each other. The tenth value, time, is positionally encoded. The time from the start of the dwell is encoded as 8 sine-cosine pairs, with periods falling geometrically from 220 ms to 0.2 ms [11]. The fast end is about the airtime of one frame, below which

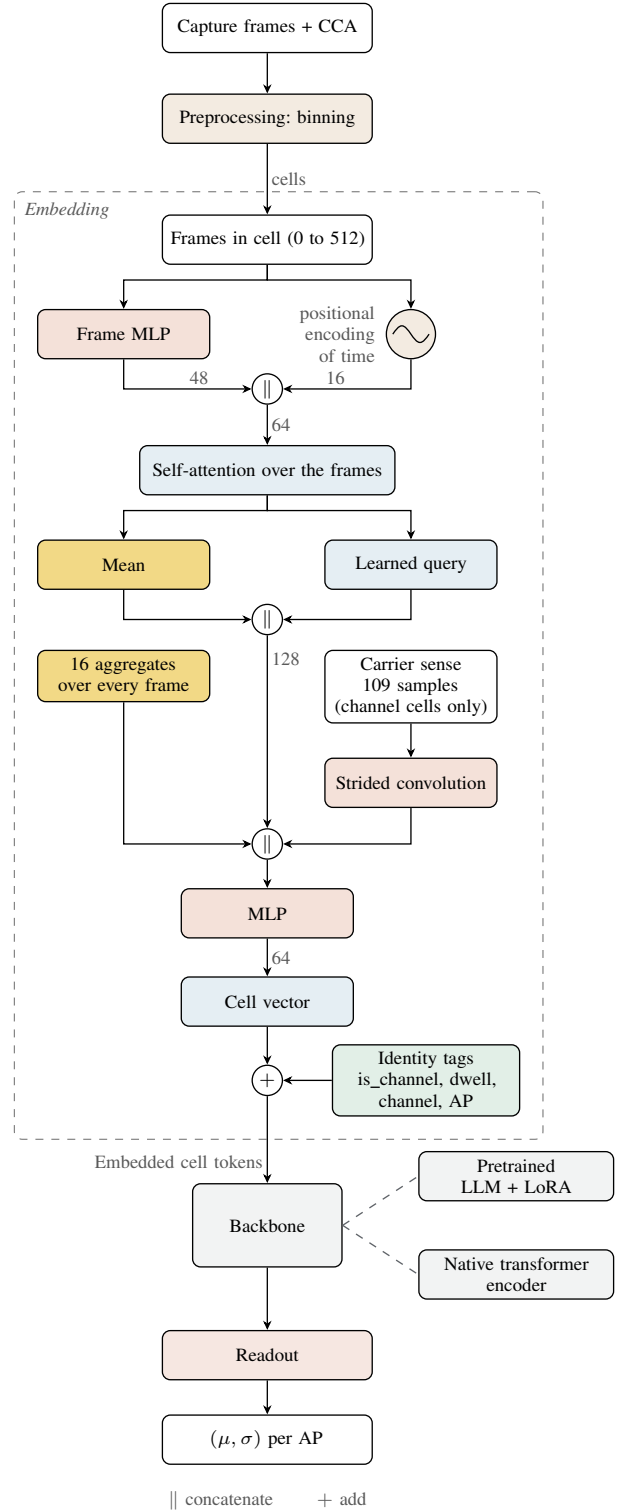


Fig. 2. Model architecture.

there is nothing to distinguish; the slow end is twice the longest gap a cell can hold, so that the sine-cosine pair turns less than half a circle over any gap within a dwell and orders it unambiguously. The 8 pairs are appended to the aforementioned embedding, creating an overall embedding dimension of 64

for each frame.

Setting position beside the content rather than adding it in [12] is what keeps the pairwise structure intact: the dot product of two frames' 16 position columns depends only on the gap between their start times, so the attention layer that follows can read how far apart two frames were without a learned layer having had to preserve the encoding.

The attended frames are then reduced to a single vector. We take two reductions and concatenate them: the mean over the cell's frames, and the output of one learned query attending over them, the pooling of the Set Transformer [13]. The mean reports the cell as a whole, while the query is free to weight a few frames above the rest. A cell hands over at most 512 frames — a random subset while training, an even stride at evaluation — which bounds an attention cost that grows with the square of the frame count; the cap binds on 0.5% of cells.

Two exact summaries join the pooled vector. The first is 16 aggregates taken over every frame the cell held, including those the cap dropped. Table II lists the quantities that are aggregated here. Such counts and totals cannot be reported by the pooled set of vectors and are thus directly given to the model. The second summary belongs to channel cells alone: the 109 carrier-sense samples of the dwell, one per millisecond, read by a strided one-dimensional convolution. (One CCA sample is lost to radio retune)

A final MLP takes the concatenation to the 64 numbers that leave the encoder (Figure 2). Both kinds of cell share the frame embedding and the self-attention, but each keeps its own pooling query and its own output MLP, since only a channel cell carries carrier sense and an aggregate such as beacon level does not mean the same thing on the two. A cell that heard nothing has nothing to pool and takes a learned vector in place of both reductions.

Nothing in the cell vector so far says which cell it is. It is built from content alone, so two cells that heard similar traffic come out alike whether they are one AP on two passes of the sweep or two different APs on different channels. Everything the attention layers do next depends on telling those apart: the thirteen cells an AP accumulates over the sweep have to find one another, an AP cell has to find the channel cell of the dwell it sits in, and the readout has to be able to ask for one AP in particular.

Identity is therefore added to the finished vector rather than mixed into it, as learned vectors for the kind of cell, for the dwell it covers and for the channel it was heard on. Every AP cell also carries a slot: an arbitrary index standing for the AP it belongs to, drawn afresh for each sample. The slot is what ties an AP's cells to each other and to the query that scores it, while keeping any address the model might memorise out of its reach. How the slot enters the token is the one part of this that differs between the two models, and is described with them. The tagged cells, 91 per scan at the median, are what the backbone reads.

V. RESULTS

A. Baselines

Three passive baselines are evaluated alongside random selection:

- *Strongest RSSI* ranks options by mean beacon level.
- *Least busy channel* ranks them by the clear-channel-assessment busy fraction of the channel each occupies, so options sharing a channel are indistinguishable to it; tied options are scored at their mean throughput.
- *RSSI minus busy* maps RSSI from -100 to -60 dBm onto the unit interval and subtracts the busy fraction from it.

TABLE I
TOP-1 ACCURACY, MEAN AND STANDARD DEVIATION OVER FIVE PARTITIONS

Model	Obj.	Top-1
<i>Heuristics</i>		
Random choice	—	0.282 ± 0.002
Least busy channel	—	0.442 ± 0.072
Strongest RSSI	—	0.647 ± 0.042
RSSI minus busy	—	0.867 ± 0.038
<i>Frame trace</i>		
Transformer [†]	reg.	0.888 ± 0.039
Transformer [†]	rank	0.897 ± 0.028
<i>Binned sequence</i>		
Ridge, all bins	reg.	0.768 ± 0.030
MLP, bin statistics	rank	0.821 ± 0.035
GBM, bin statistics	reg.	0.842 ± 0.029
GBM, all bins	reg.	0.845 ± 0.020
MLP, all bins	rank	0.847 ± 0.032
Transformer	reg.	0.873 ± 0.026
Transformer	rank	0.873 ± 0.037
<i>Binned sequence, averaged over time</i>		
MLP	rank	0.767 ± 0.039
GBM	reg.	0.803 ± 0.059
<i>Vector: 35 hand-built features</i>		
Ridge	reg.	0.881 ± 0.045
MLP, pointwise	rank	0.897 ± 0.024
Set ranker	rank	0.909 ± 0.030
Random forest	reg.	0.925 ± 0.019
Gradient boosting	reg.	0.934 ± 0.020
<i>Vector and binned sequence</i>		
Transformer	reg.	0.919 ± 0.021
Transformer	rank	0.931 ± 0.030

VI. INTERPRETATION AND NEXT STEPS

REFERENCES

- [1] P. Dely, A. Kassler, L. Chow, N. Bambos, N. Bayer, H. Einsiedler, and C. Peylo, "BEST-AP: Non-intrusive estimation of available bandwidth and its application for dynamic access point selection," *Computer Communications*, vol. 39, pp. 78–91, 2014, doi: 10.1016/j.comcom.2013.11.006.
- [2] T. Adame, M. Carrascosa, B. Bellalta, I. Pretel, and I. Etxebarria, "Channel load aware AP/Extender selection in home WiFi networks using IEEE 802.11k/v," *IEEE Access*, vol. 9, pp. 30095–30112, 2021, doi: 10.1109/ACCESS.2021.3059473.
- [3] W. Wong and S.-H. G. Chan, "Distributed joint AP grouping and user association for MU-MIMO networks," in *Proc. IEEE INFOCOM*, 2018, pp. 252–260, doi: 10.1109/INFOCOM.2018.8486284.

- [4] Y. Zhang and L. Dai, “Joint optimization of placement and coverage of access points for IEEE 802.11 networks,” in *Proc. IEEE ICC*, 2020, pp. 1–7, doi: 10.1109/ICC40277.2020.9149098.
- [5] IEEE 802.11 Task Group ax, “TGax simulation scenarios,” IEEE 802.11-14/0980r16, 2016. [Online]. Available: <https://mentor.ieee.org/802.11/dcn/14/11-14-0980-16-00ax-simulation-scenarios.docx>
- [6] 3GPP, “Evolved Universal Terrestrial Radio Access (E-UTRA); Further advancements for E-UTRA physical layer aspects,” TR 36.814 V9.0.0, Mar. 2010.
- [7] “Internet traffic volumes are not Gaussian — they are log-normal: An 18-year longitudinal study with implications for modelling and prediction,” arXiv:2007.10150, 2020.
- [8] ns-3 Project, *ns-3 Model Library, Release 3.45*, 2025. [Online]. Available: <https://www.nsnam.org/docs/release/3.45/models/>
- [9] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, “NetLLM: Adapting large language models for networking,” in *Proc. ACM SIGCOMM*, 2024, pp. 661–678, doi: 10.1145/3651890.3672268.
- [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. ICLR*, 2022.
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [12] G. Ke, D. He, and T.-Y. Liu, “Rethinking positional encoding in language pre-training,” in *Proc. ICLR*, 2021.
- [13] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, “Set Transformer: A framework for attention-based permutation-invariant neural networks,” in *Proc. ICML*, 2019, pp. 3744–3753.

APPENDIX A

MODEL INPUTS

Every number the models read, in the three groups they arrive in. Rates and fractions are normalized by the time actually spent listening on the relevant channel, not by the observation window. A `log1p` suffix marks a count or duration stored as $\log(1 + x)$.

TABLE II
THE MODEL INPUTS. FRAME FEATURES ARE READ ONCE PER DECODED FRAME, AGGREGATES ONCE PER CELL OVER EVERY FRAME IT HELD, AND DESCRIPTORS ONCE PER DISCOVERED AP OVER THE WHOLE WINDOW.

Column	Meaning
Frame	
offset_in_dwell	Where in the 110ms dwell the transmission began, 0 to 1; encoded as sine-cosine pairs rather than embedded
gap_loglp	Idle time on this channel before it
rss_i_dbm	Received level
airtime_loglp	The frame's own airtime, not that of the transmission carrying it
length_loglp	Length in bytes
rate_loglp	PHY rate
is_beacon, is_retry	Beacon and retry flags
from_ap	The BSSID itself sent it, not one of its stations
of_valid_ap	Carries a discovered AP's BSSID
category	Management, control or data; embedded rather than numeric
Cell aggregate	
frames_loglp, bytes_loglp	Frames and bytes the cell held
airtime_fraction	Their own airtimes summed, over the time listened
busy_fraction	Time with a decoded transmission on air, overlaps counted once
data_fraction, mgmt_fraction	Share of frames by category
retry_fraction	Share marked retry
transmitters_loglp	Distinct transmitters; on an AP cell its own is left out
beacons_loglp	The AP's beacons; 0 on a channel cell
beacon_rssi_mean	Their level; missing on a channel cell
rssi_mean, rssi_max	Over every frame, its stations' as well as the AP's
data_rate_loglp	Mean PHY rate of the data frames
longest_gap_fraction	Longest stretch with nothing decoded, over the time listened
cca_mean, cca_max	Carrier sense over the dwell; 0 on an AP cell
AP descriptor	
beacon_rssi_{mean, std, min, max, last}	Beacon level over the whole window: mean, spread, extremes, and most recent
beacons_loglp	Beacons decoded
bss_airtime_fraction	Its BSS's summed airtime, over the time listened on its channel
bss_transmitters_loglp	Distinct stations heard sending in its BSS
cochannel_valid_aps	Other discovered APs on its channel
n_valid_aps	Discovered APs in the scan